AssetTrust v0.50

Confidential document metadata row-level security - engineering assessment

RELEASE DECISION: NO-GO. This is an offline-tested development checkpoint, not an approval to host real customer evidence or claim official land verification.

Executive summary

The v0.50 code adds PostgreSQL FORCE ROW LEVEL SECURITY on the private **documents** metadata table. It builds on six previously scoped tables, bringing the source-level inventory to seven. The professional credential application now scopes its supporting-document lookup inside the existing SQL transaction.

What was changed

- Migration 025: an additive, default-deny tenant policy for document rows, with read/write scope linked to a transaction-local validated organization UUID.
- Production database startup: requires enabled and forced RLS across all seven named tables, and refuses privileged runtime roles or ownership of those tables.
- Professional supporting-document verification: tenant context is established before checking organization and uploader permissions.
- CI and safety: new version tests, a guarded two-tenant PostgreSQL contract, and a read-only SQL inventory.

Verified offline

Verification	Result
New v0.50 tests	9/9 PASS
Version scripts v0.03-v0.50	48/48 PASS
SQL migration fingerprints	25/25 MATCH
TypeScript/TSX syntax scan	102 files; 0 diagnostics
Architecture and static CI coverage	PASS
Snapshot source scanner	1,981 files; 0 high-confidence matches

Passing an offline or syntax test does not prove real PostgreSQL policies, authenticated browser behavior, dependency safety, or infrastructure security.

Security boundaries and outstanding proof

What this milestone protects

The **documents** table stores organization, related asset/case IDs, upload metadata, file size, digest, review state and encryption references. Tenant RLS protects these database rows from a properly restricted SQL role. Encrypted blob bytes remain protected by separate application/storage controls and do not become safe merely because metadata has RLS.

Remaining important limitations

- **Live database tests:** migration 025 and its guarded PostgreSQL contract have not been executed here. Neither cross-tenant denial nor correct pool reuse is proven on a real database.
- **Build gate:** the source archive has no authentic package-lock.json or installed workspace dependencies. npm ci failed as expected; no complete backend/frontend build was proven.
- **Remaining data:** users, audit events, professional accreditation and other operational tables are not covered by this migration. Full database-wide isolation is not established.
- **External attack surface:** HTTPS/HSTS, secure cookies, CSRF, firewall/SSH configuration, recipient access concurrency, malware scanning and encrypted backups still require real deployment checks.
- **Secrets:** the source-snapshot scan returned no high-confidence matches, but the full canonical Git history was unavailable. No secret was rotated during this checkpoint.
- **Legal trust:** internal verification reports are not government certificates or proof of land ownership.

Release preflight

3 of 11 prerequisites present. Node runtime, static CI coverage and migration fingerprints are present. Reproducible dependency installation, compiled artifacts, environment configuration, live PostgreSQL integration and production security evidence remain outstanding.

Why RLS must be staged

PostgreSQL RLS with a transaction-local tenant setting is only one layer. The application must still validate user permissions, constrain cross-tenant foreign-key relationships, use parameterized SQL and connect with a non-owner/non-BYPASSRLS database role. Previous API versions may not set tenant scope for new document reads; mixing them with migration 025 can deny legitimate document checks.

Required Kali Linux validation

Before changing a real database

- Verify a full database backup by restoring it separately; preserve encryption and signing keys in a secure secret manager.
- Generate, inspect and commit a real npm lockfile in the original Git repository; run npm ci, npm audit, npm test and npm run build without bypasses.
- Run the existing schema and all migrations in a disposable PostgreSQL/PostGIS database named *_ci or *_test on loopback, then validate migration 025 and run the new document RLS contract.
- Test two organizations and roles end-to-end: document listing, upload, download, denied cross-tenant actions, professional support files, dashboards and pool reuse.
- Run full Git-history secret scanning, verify HTTPS/SSH/firewall and browser security, test upload malware detection and backup recovery, and complete an independent adversarial assessment.

Essential commands

```
npm install --package-lock-only

npm ci && npm audit --omit=dev --audit-level=high

npm run verify:v50 && npm run verify:ci:v41

npm run verify:architecture && npm run verify:syntax

npm test && npm run build

# ONLY on approved local disposable PostgreSQL/PostGIS:

export NODE_ENV=test

npm run db:migrate && npm run db:check

psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f tools/sql/v50-documents-rls-readonly-audit.sql

ASSETTRUST_E2E_ALLOW_MUTATION=YES npm run test:pg:documents-rls:v50
```

Do not apply migration 025 while older API instances serve the professional-document path. Do not use the disposable test against production. The test requires a suitably privileged migration/test operator, never the production runtime account.

Next engineering priority

Clear the authentic dependency/build gate and run real PostgreSQL tests before continuing the database-wide tenant-isolation rollout. The next RLS candidate requires a separate policy design for user memberships and administrative oversight.

Decision: HOLD RELEASE until every critical/high finding is resolved, the live checks pass, and an authorized reviewer signs off.